AI banking assistant design

Oct 26, 2026 · @Kevin

Summary

A chat panel in the web app lets a signed-in customer ask about their own finances in plain English. Claude, from Anthropic, answers by calling two read-only tools that wrap the existing AccountService, always for the signed-in customer. The assistant can't move money or change anything, because none of its tools write.

Example questions from the requirements:

- "What is my checking balance?"
- "How much did I spend on groceries this month?"
- "Show my last five transactions."

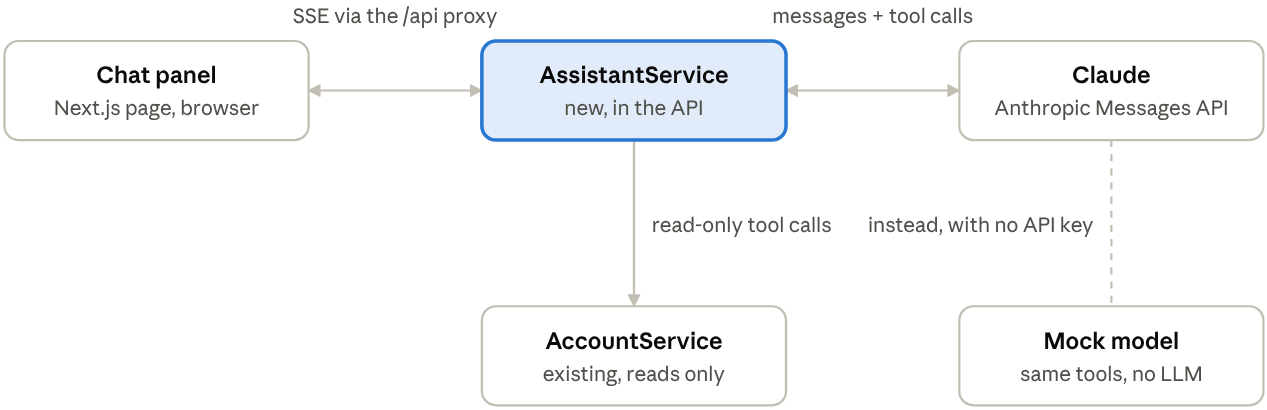
Without an Anthropic API key, a built-in mock mode answers those same kinds of question from the same real data. The panel works with no model access, and the tests use the mock mode too.

Decided on 2026-10-26: the assistant is read-only, and the call to Claude lives in the ASP.NET Core API. The Next.js app stays client-only and holds no secret.

Architecture

The API gets one new controller, one service and two tool classes. The browser talks to it through the existing /api proxy, so nothing new needs CORS or a second secret.

Every answer comes from read-only calls for the signed-in customer



assistant request flow · 5 parts

The assistant can only reach AccountService's read methods. It has no route to TransferService, so it has no way to move money.

- **AssistantController** (thin) takes `POST /api/v1/assistant/messages` and streams the reply as server-sent events. It requires the session like every other controller.
- **AssistantService** owns the conversation and the tool loop. It sends the history to the model, runs each tool call with the session's customer id, and repeats until the model ends its turn. It's a hand-written loop, so each text chunk can go straight to the stream.
- **IAssistantModel** has two implementations. `ClaudeAssistantModel` uses the official C# SDK (`dotnet add package Anthropic`). `MockAssistantModel` matches a few question patterns (balance, spending in a period, last N transactions), calls the same tools, and fills in a template answer. The mock is used when no API key is set or when `Provider` is `Mock`.
- **Tools** are small classes under `Assistant/Tools/`. Each one validates its input, calls AccountService, and returns compact JSON. They never touch a repository or the DbContext directly, which keeps the architecture tests passing.
- **Conversation state** is held server-side in `IMemoryCache`, keyed by session id and conversation id, with a 30-minute sliding expiry. It's cleared on logout. The client sends only the new user message.

Tools

The model gets two tools, and both only read. Neither one takes a customer id: the service passes the session's customer id to every call, so the model can't ask about anyone else's accounts.

Tool	Calls	Returns
`list_accounts`	AccountService.GetAccountsAsync	Each account's id, nickname, masked number, type and balance
`search_transactions`	AccountService.GetHistoryAsync	Up to 100 transactions for one account, newest first. Optional filters are from/to dates, debit or credit, and text search, plus a cursor for the next page.

How the three example questions are answered:

Question	Tool calls
What is my checking balance?	`list_accounts`, then pick the checking account
How much did I spend on groceries this month?	`list_accounts`, then `search_transactions` with type debit, from the 1st of this month to today. The model picks the grocery merchants and adds up their amounts.
Show my last five transactions.	`list_accounts`, then `search_transactions` with a limit of 5 on the checking account (or the account named)

Notes:

- "Groceries" isn't a category in the data. The ledger's categories are coarse (card, payment, deposit and so on), so grocery spending is found from merchant names such as FreshMart Grocer. Claude judges this from the names. The mock mode uses a fixed keyword list (`grocer` , `market` , `supermarket`), which is enough for the seeded merchants.

- "This month" uses the bank's time zone (America/New_York), like the existing date filters.
- Every tool uses `strict: true` and a JSON schema with `additionalProperties: false`, so the arguments always match. Amounts are decimal strings in USD, as in the REST API.
- A tool error, such as an unknown account, returns the same problem detail the REST API would. The model then explains the problem in plain words.

API contract

There are two new endpoints, both of which require the session, and they're added to openapi.yaml.

Method and path	Body	Response
`POST /api/v1/assistant/messages`	`{ "conversationId": "uuid or null", "text": "..." }`	`text/event-stream` (events below)
`DELETE /api/v1/assistant/conversations/{id}`	none	204. It starts a new chat.

Stream events:

Event	Data
`conversation`	`{ "conversationId": "...", "mode": "claude" or "mock" }`, sent first
`status`	`{ "tool": "search_transactions" }`, so the UI can show "Looking up your transactions..."
`text`	`{ "delta": "..." }`, a chunk of the reply
`done`	`{ "stopReason": "end_turn" }`
`error`	an ApiProblem: the model is unavailable, or the conversation expired

Errors before the stream starts use the usual problem+json responses: 401 with no session, and 400 for empty text.

The generated RTK Query client can't consume SSE, so the front end calls this endpoint with `fetch` and a stream reader. The DELETE endpoint goes through RTK Query as usual.

Front end

The assistant is a slide-over chat panel opened from an **Ask** button in the app header, so it's available on every signed-in page.

- **State:** a new `assistantSlice` holds the conversation id, the messages shown, and a streaming flag. It isn't persisted, so a page reload starts a new chat.
- **Streaming:** `src/features/assistant/streamAssistant.ts` posts with `fetch`, reads the event stream, and dispatches an action per event. A 401 dispatches the existing `sessionEnded` action.
- **Rendering:** replies are shown as plain text with simple Markdown (lists, bold). No HTML from the model is ever rendered. When the mode is mock, a small "Demo answers" label shows under the panel title.
- **Accessibility:** the message list is an `aria-live="polite"` region, focus moves to the input when the panel opens, and Escape closes it. The axe check runs on the open panel.
- **Browser-only demo mode:** with `NEXT_PUBLIC_API_MOCKING=enabled`, an MSW handler replays a canned stream.

Safety and privacy

Read-only is enforced by the structure, not by the prompt. The model can only call two read methods for one customer, so no wording of a question or of a transaction description can make it change anything.

- **Customer scoping:** the customer id comes from the session claims in AssistantService, never from tool input. A test asks about another customer's account id and expects a not-found result.
- **Prompt injection:** transaction descriptions are data that someone else could write. They reach the model only inside tool results, and at worst they could change the wording of a reply.
- **System prompt:** it says the assistant serves one signed-in customer of a demo bank, is read-only, uses tools for every figure instead of guessing, and declines requests to move money with a pointer to the Transfers page. It's a fixed string, so it caches well.

- **Data sent to Anthropic:** account nicknames, masked numbers, balances and transactions, but never passwords, session tokens or full account numbers. This is fine for the seeded demo data. A real deployment would need a data-processing review first.
- **Logging:** the API logs tool names, token usage and latency, but not message text.

Model and configuration

The default model is Claude Opus 5.5 (`claude-opus-5-5`), Anthropic's current default model. It runs at effort `low` , which suits short chat answers. Both are settings, so switching to Claude Sonnet 5.5 (`claude-sonnet-5-5` , about half the price) is a config change.

Setting (`Bank:Assistant`)	Default
`Provider`	`Claude` , or `Mock`
`Model`	`claude-opus-5-5`
`Effort`	`low`
`MaxTokens`	4096
`MaxToolRounds`	6. After that, the turn ends with "I couldn't finish that, please try a narrower question."

- **API key:** `ANTHROPIC_API_KEY` comes from `dotnet user-secrets` in development and from an environment variable elsewhere. It's never stored in appsettings or the repo. If no key is set, the API logs one warning at startup and uses the mock model, so the app and the existing tests run with no model access.
- **Caching:** the system prompt and the two tool definitions are cached between turns.
- **Cost (approximate):** a typical question with two tool calls is roughly 6,000 input and 300 output tokens, or about 3 cents on Opus 5.5 before caching. That's fine for a demo.

Testing

The automated tests never call the real model. They swap a scripted `IAssistantModel` into the factory, which returns set tool calls and text, so the runs are free and repeatable.

- **API integration tests** (xUnit, real PostgreSQL, the existing BankApiFactory): each tool returns the right data for alice; a call for ben's account id while signed in as alice returns not-found; the stream events arrive in the right order; 401 and 400 work; and the conversation is gone after logout.
- **Mock model tests:** the three example questions get correct answers from alice's seeded data, including the grocery total for the current month.
- **Read-only checks:** after a chat, the ledger and transfer row counts are unchanged. A new architecture test checks that nothing under `Assistant/` references TransferService or any repository. The existing three architecture tests still pass.
- **Front end:** Vitest + MSW cover the stream parser and the slice. A Playwright test opens the panel, asks the three example questions against the real API in mock mode, and runs an axe check.
- **Live smoke test:** a manual script, not run in CI, asks the three example questions against the real model with your key.

Beyond the requirements, and open questions

A few small items go beyond the requirements. Each one is easy to drop:

- [ ] The streamed reply and the "Looking up your transactions..." status. A single JSON reply would be simpler, but it would show nothing for several seconds.
- [ ] The `MaxToolRounds` cap and the 30-minute conversation expiry. These are guards against runaway cost and memory.
- [ ] The "Demo answers" label in mock mode.

Left out on purpose, and easy to add later: payees and past transfers as tools, a rate limit on assistant messages, and chats that survive a reload.

There are no open questions.